

# Mini Hiring Pipeline

A single-recruiter hiring pipeline: a linear, unbreakable stage machine, an audit trail the database itself refuses to alter, and one search box that takes plain English.

<https://github.com/artzueros/mini-hiring-pipeline>

## The pipeline

Candidates move forward exactly one stage at a time. Skipping and reversing are impossible - not validated against, but unrepresentable: one pure function decides every legal move, and the rest of the system asks it.



Every move, including the initial creation, appends a row to an immutable history. Immutability is enforced by BEFORE UPDATE / BEFORE DELETE triggers in Postgres, so it holds even against a direct psql session - not merely because the application promises not to.

## The search box

Deterministic rules parse the query first; a language model is consulted only for phrasing the rules genuinely cannot read. Every example from the brief is handled by the rules alone, with no API key and no network call:

```
"Find Priya Sharma" / "sharam"      -> Priya Sharma      (typo tolerant)
"Who's in Interview right now?"      -> currently in that stage
"stuck in Screening for more than a week" -> in stage at least that long
"Who moved to Interview since Monday?" -> entered that stage on/after a date
"reached the Offer stage, not hired"   -> reached it, currently elsewhere
"Everyone except rejected candidates." -> excludes rejected, KEEPS hired
```

Criteria combine and results rank best-match-first. When a query cannot be understood the caller gets a 422 naming what defeated the parser and a list of queries that work - never an empty list, which would be indistinguishable from a successful search that found nobody.

Note text is searched too, but only after names and after the model: a note reading "this is a goat" is found by searching goat. The report that prompted this blamed the name threshold; the real cause was that note text was never searched. Sitting last, the fallback can only turn a 422 into a 200 - no answer that already worked can change.

## Architecture

|                       |                                                                 |
|-----------------------|-----------------------------------------------------------------|
| HTTP                  | api/ (JSON)      web/ (HTML board + search box)                 |
| Services              | candidate_service   search: rules -> executor -> llm_fallback   |
| Repositories / Models | candidate_repo   SQLAlchemy ORM                                 |
| Domain                | domain/pipeline.py - pure state machine, no DB, no HTTP, no I/O |

|        |                                                                                                                  |
|--------|------------------------------------------------------------------------------------------------------------------|
| Stack  | Python 3.12, FastAPI, SQLAlchemy 2 async + asyncpg, Alembic, Postgres 16 (pg_trgm), Pydantic v2, Jinja2, pytest. |
| Tests  | 261 passing. None touch the network; the model fallback runs against a scripted fake.                            |
| Run it | docker compose up --build, or ./scripts/db.sh start + alembic upgrade head + uvicorn.                            |

## Five decisions, and why

- Immutability is a database trigger, not a convention. An audit trail that holds only as long as nobody opens psql is not an audit trail. The cost was switching test cleanup from DELETE to TRUNCATE, which does not fire row triggers.
- Recruiter notes are a second append-only table with their own trigger function, not extra rows in the audit trail. A note is not a move, and a shared table would have forced to `_stage` to become nullable -- at which point "who reached Offer" starts matching rows that moved nobody anywhere.
- One clock: Postgres `now()`, never Python. `now()` is the transaction timestamp, so writing the stage change and its audit row in one transaction makes them equal by construction. A test asserts exact equality.
- Time in stage is derived on every read, never stored. A stored duration is wrong the moment it is written and silently drifts from the history.
- Rules before the model, for determinism rather than cost. A query that parses the same way every time is one whose bugs reproduce and whose fixes can be verified.
- `pg_trgm` rather than a fuzzy-matching library: the database already does it, with an index, instead of loading every name into memory to compare in Python.
- Note search matches whole words, not trigrams. `similarity('goat', 'goal')` is 0.429 - above the name threshold - so a fuzzy note search would answer "goat" with everyone who wrote down a goal.

## Where I disagreed with the AI

The name matcher I wrote first - scoring the whole name with Postgres trigram similarity - passed the brief's single example, "sharam" -> Sharma. I then measured it against a wider matrix instead of trusting it, and it was not merely imprecise; it was ordered wrongly:

```
similarity('sharam', 'Priya Sharma') 0.57 <- the candidate
similarity('sharam', 'Vikram Singh') 0.43 <- not the candidate
similarity('shrma', 'Priya Sharma') 0.44 <- the candidate
similarity('shrma', 'Fatima Sheikh') 0.50 <- not the candidate
```

"shrma" scored a non-candidate above the real match. No threshold fixes that - the ordering itself was broken, because a short query against a long name is dominated by trigrams the two share by coincidence ('sharam' and 'Singh' both contain s and h).

I replaced it with word-against-word comparison: split both sides, take the best similarity between any query word and any name word, average across query words. Every false positive disappears and the weakest true match (0.40) sits clear of the strongest false one (0.25). The 0.35 threshold is set from that gap, not from taste.

I gave up something real for it: the score is computed per row, so the GIN trigram index can no longer serve the filter. At this scale that is unmeasurable, and it buys a matcher that is actually correct. Both cases are pinned by tests naming the bug they exist to prevent.

Two smaller disagreements: the plan said "API only, no frontend" where the brief asks for a web app with a search box, so the API was built first and completely and one server-rendered page was added on top; and the plan's `next_stage()` had an unreachable branch and could not express a skip or a reversal as a call, making its own highest-value tests unwritable at the domain layer.

## With more time

- Optimistic concurrency on stage moves. Two recruiters advancing one candidate at once produce two audit rows for one real move. A version column with a conditional UPDATE would turn that into a detectable conflict.
- A golden-file suite of several hundred real recruiter phrasings, so grammar changes show up as diffs rather than surprises.
- Transposition typos ("shrama" for "sharma") currently fall below threshold and return an explanatory 422 suggesting a spelling check. Trigram similarity is weak on transpositions by construction; a Damerau-Levenshtein pass on tokens would catch them.
- Pagination on GET /candidates. The board already eager-loads each candidate's transitions and notes, which is one extra query per load at this scale and would not be at a few thousand.

## Deployment

Not deployed. The brief's deliverable list does not include a live URL, and the container setup is written but unverified - Docker is not installed on the machine this was built on, and the README says so rather than implying otherwise. The local path is the one that has been run end to end, with the full suite against it. A `docker-compose.yml`, `Dockerfile` and a migration-then-serve entrypoint are in the repo for review.